

第8章: 編集・削除・詳細表示機能の実装

前章で作った「一覧表示（Read）」と「新規登録（Create）」に続いて、この章では残りの機能を全て実装します。この章を終えると、**CRUDの全機能が揃った完動するアプリ**が完成します！

この章で実装する機能

前章までで、書籍の一覧表示と新規登録機能を実装しました。この章では、CRUD（Create=作成, Read=読み取り, Update=更新, Delete=削除）の残りの機能を実装します。

機能	CRUDのどれ？	ユーザーの操作	技術的に行うこと
詳細表示	R（Read）	書籍カードをクリック → 全情報を表示	動的ルーティング（ <code>/books/[id]</code> ）で Supabase から1件取得
編集	U（Update）	詳細画面の「編集」ボタン → フォームで修正 → 保存	既存データをフォームに表示し、Supabase の UPDATE を実行
削除	D（Delete）	「削除」ボタン → 確認ダイアログ → 削除	Supabase の DELETE を実行し、一覧に戻る
検索・フィルタ	（発展）	キーワード入力やステータスで絞り込み	Supabase の <code>ilike</code> （部分一致検索）クエリを使用
ソート	（発展）	「新しい順」「評価順」などで並べ替え	Supabase の <code>order</code> クエリを使用

動的ルーティングとは？ URLの一部を変数として扱う仕組みです。例えば `/books/abc123` と

`/books/xyz789` は、同じページ定義（`/books/[id]/page.tsx`）で処理されます。`[id]` の部分がそれぞれ `abc123` や `xyz789` に置き換わります。「URLの中の変わる部分」を **動的セグメント（dynamic segment）** と呼びます。セグメントとは URL を `/` で区切った 1 区画のことで、`/books/abc123` なら `books` と `abc123` がそれぞれ 1 セグメントになります。

HTTP メソッドのざっくり整理: ブラウザがサーバーに対して行う操作には種類があり、これを HTTP メソッドと呼びます。CRUD と対応付けると次のようになります。 - **GET:** ページやデータを「読む」（Read）。一覧表示や詳細表示で使う。何度実行しても結果が変わらない。 - **POST:** 新しいデータを「作る」（Create）。フォーム送信などで使う。実行のたびに新しいレコードが増える。 - **PUT / PATCH:** 既存データを「書き換える」（Update）。同じリクエストを 2 回送っても結果が同じ（これを **冪等性（べきとうせい）** と呼びます）。 - **DELETE:** データを「消す」（Delete）。一度消したものをもう一度消そうとしても、すでに無いので結果は同じ（これも冪等）。

Next.js では、これらを意識せずに JavaScript の関数として呼び出せる場面が多いですが、裏側ではこの 4 つのいずれかが走っています。

目次

0. 前提知識: 動的ルーティング・useState・useRouter

1. 詳細表示機能
 2. 編集機能
 3. 削除機能
 4. 検索・フィルタ機能（発展）
 5. ソート機能
 6. 全機能の動作確認
 7. トラブルシューティング
 8. 全体のページ遷移図
-

0. 前提知識: 動的ルーティング・useState・useRouter

この章では、特定の本を「ID指定で開く・編集する・削除する」機能を作ります。コードに入る前に、3つの仕組みだけ押さえます。

0.1 動的ルーティング（Dynamic Routes）

「URLの中に変数を含めたい」場合に使う Next.js の機能です。

```
app/books/[id]/page.tsx ← フォルダ名を【角括弧】で囲む
```

このファイルがあると、

アクセスされたURL	内部で <code>params.id</code> の値
<code>/books/abc123</code>	<code>"abc123"</code>
<code>/books/xyz789</code>	<code>"xyz789"</code>
<code>/books/42</code>	<code>"42"</code>

のように、`[id]` の部分が変数として取り出せます。Next.js 15 以降では `params` は `Promise` で渡されるため、`await` で取り出す必要があります（後ほどコード内で出てきます）。

なぜ Promise なの？ Next.js 15 から、ページコンポーネントの `params` や `searchParams` は「すぐには値が確定していないかもしれない非同期データ」として扱われるようになりました。Promise とは「あとで値が決まる箱」のようなもので、`await` を付けることで「箱から値が出てくるのを待つ」ことができます。Next.js 14 以前は普通のオブジェクトだったため、移行時にこの違いでつまづきやすいので注意してください。

0.2 `useState` の超復習

第3章で出てきた React Hook。「コンポーネントが覚えておきたい状態（変化する値）」を持つために使います。

▼ このコードがやること（先に日本語で）：ボタンを押すたびに数字が増える、シンプルなカウンターを作ります。カギになるのは `useState` で、「画面に覚えさせておきたい値（ここではカウント数）」と「その値を書き換える専用の関数」をセットで受け取ります。初心者はず「値を直接書き換えるのではなく、更新用の関数を呼ぶと画面が描き直される」という流れだけ押さえてください。各行の詳しい意味はコード内のコメントにあります。

```
"use client"; // このファイルはクライアント側（ブラウザ）で動かす宣言。これがないと useState などのフックは使えない
import { useState } from "react"; // React から useState フックを取り込む

export default function Counter() { // Counter という関数コンポーネントを定義し、デフォルトエクスポートする
  // [現在の値, 値を更新する関数] = useState(初期値)
  // 配列分割代入で 2 つの値を一度に受け取る書き方
  const [count, setCount] = useState(0); // count の初期値は 0、setCount で値を更新する

  return (
    <div>
      <p>カウント: {count}</p> {/* 波括弧 { } の中に JavaScript の値を書くと、その値が描画される */}
      <button onClick={() => setCount(count + 1)}>+1</button> {/* クリック時に count を +1 する関数を渡す */}
    </div>
  );
}
```

▼ 動作: - 最初に画面に「カウント: 0」と表示される - ボタンを押すたびに count が +1 され、画面が再描画される

0.3 `useRouter` で画面遷移

ボタンを押した後にプログラムの別ページに遷移したいときは `useRouter` を使います。`<Link>` は「ユーザーがクリックして遷移する」ためのもの、`useRouter` は「コードが勝手に遷移させる」ためのもの、と覚えると分かりやすいです。

▼ このコードがやること（先に日本語で）：ボタンを押したら、JavaScript のコードから別ページ（ `/books` ）へ自動で移動させる例です。`useRouter` で取得した `router` オブジェクトの `push` を呼ぶと、リンクをクリックしたのと同じように画面が切り替わります。初心者は「リンクは人がクリック、`router.push` はプログラムが移動させる」という違いだけ覚えておけば十分です。各メソッドの使い分けはこのあとの表とコメントで補足します。

```
"use client"; // useRouter はクライアントコンポーネント専用なので、この宣言が必須
import { useRouter } from "next/navigation"; // App Router 用の useRouter は
next/navigation から取る (旧 next/router ではない)

export default function MyComponent() {
  const router = useRouter(); // ルーター（ページ遷移を司るオブジェクト）を取得

  const handleClick = () => { // ボタンクリック時に呼ばれる関数
    router.push("/books"); // /books に遷移する（履歴に積まれる＝戻るボタンで戻れる）
    router.refresh(); // 現在ページのサーバー側データを最新化（必要時のみ呼び）
  };

  return <button onClick={handleClick}>一覧へ戻る</button>; // クリックで handleClick を実行
}
```

▼ よく使うメソッド:

メソッド	用途	例
<code>router.push("/books")</code>	新しいURLに遷移	削除後に一覧へ戻る
<code>router.replace("/login")</code>	履歴を残さず遷移	ログイン誘導
<code>router.back()</code>	ブラウザの戻るボタンと同じ	キャンセル
<code>router.refresh()</code>	現在ページを再取得	データ更新後の反映

0.4 Server Component と Client Component の使い分け（おさらい）

種類	デフォルト/宣言	できること	できないこと
Server Component	デフォルト（何も書かない）	DBに直接アクセス、 <code>await</code> で取得	<code>useState</code> / イベントハンドラ
Client Component	先頭に <code>"use client";</code> を書く	<code>useState</code> 、ボタンの <code>onClick</code> など	DB直接アクセス（API経由になる）

この章での使い分け: - 詳細ページ（読み込みのみ） → Server Component - 編集フォーム（入力に反応させる） → Client Component - 削除ボタン（確認ダイアログ + APIコール） → Client Component

フォームの送信フローについて補足: 一般的に Web のフォームは「ユーザーが入力 → 送信ボタン → ブラウザがサーバーに POST → サーバーが処理 → 結果ページを返す」という流れで動きます。この章のフォームはちょっと違って、「ユーザーが入力 → 送信ボタン → JavaScript が `e.preventDefault()` で標準送信をキャンセル → Supabase クライアント経由で直接 DB に書き込み → `router.push` でページ遷移」という SPA 的な流れになっています。標準送信ではないので、ページがリロードされず、画面のチラつきがありません。

ブラウザのキャッシュと再送信について: フォーム送信後にブラウザの「更新」ボタンを押すと、「フォームを再送信しますか?」というダイアログが出る場合があります。これは「同じ POST をもう一度送ろうとしている」状態で、二重登録の原因になります。本章のように `router.push` で別ページに遷移すると、再読み込みしても遷移先のページが更新されるだけで、フォームは再送信されません（これを Post/Redirect/Get パターンと呼びます）。

1. 詳細表示機能

書籍の詳細情報を表示するページを作成します。ここでは Next.js の動的ルーティング（Dynamic Routes）を活用します。

1-1. 動的ルーティングとは

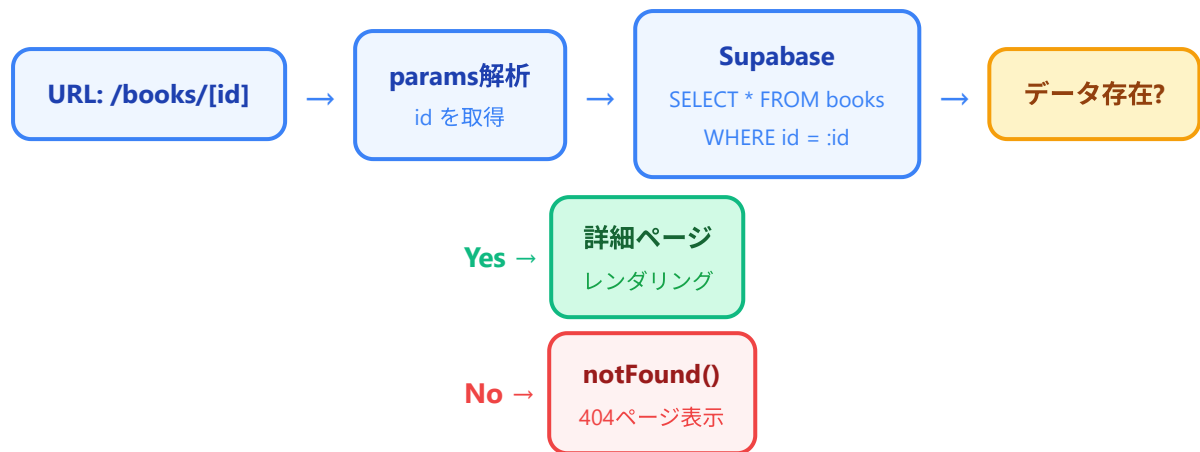
Next.js App Router では、フォルダ名を **【パラメータ名】** のように角括弧で囲むことで、動的な URL セグメントを作成できます。例えば `app/books/[id]/page.tsx` というファイルを作ると、`/books/abc123` や `/books/xyz789` のような URL にマッチし、`abc123` や `xyz789` の部分を `params.id` として取得できます。

```
app/
  books/
    page.tsx      → /books（一覧ページ）
    new/
      page.tsx    → /books/new（新規登録ページ）
    [id]/
      page.tsx    → /books/:id（詳細ページ）
      edit/
        page.tsx  → /books/:id/edit（編集ページ）
```

角括弧フォルダの優先順位: `/books/new` と `/books/[id]` の両方が存在する場合、Next.js は「具体的なパス（`new`）」を「動的パス（`[id]`）」より優先します。なので `/books/new` にアクセスすると `new/page.tsx` が、`/books/abc123` にアクセスすると `[id]/page.tsx` が呼ばれます。

1-2. 処理フローの概要

以下のフロー図は、詳細ページにアクセスしたときの処理の流れを示しています。



URLにアクセスすると、Next.js が URL 内の `[id]` 部分を解析して `params` オブジェクトに格納します。そのIDを使って Supabase からデータを取得し、データが存在すれば詳細ページをレンダリング、存在しなければ 404 ページを表示します。

HTTP のステータスコード: ブラウザとサーバーのやりとりには結果を表す数字（ステータスコード）が付きます。よく見るものは **200**（正常）、**401**（認証されていない＝ログインしていない）、**403**（権限がない＝ログインはしているが見る資格がない）、**404**（見つからない）、**500**（サーバー側のエラー）です。本章では「存在しない ID にアクセスされたら 404」「ログイン制御を入れるなら 401/403」と覚えておけば十分です。

1-3. 書籍詳細ページの作成

まず、`app/books/[id]/` ディレクトリを作成し、その中に `page.tsx` を作成します。

ファイル: `app/books/[id]/page.tsx`

▼ このコードがやること（先に日本語で）：URL の `/books/[id]` の `[id]` 部分を手がかりに、Supabase から書籍を1件だけ取ってきて、その全情報をカード形式で表示するページです。このページはサーバー側で動く Server Component なので、`await` を使ってデータベースから直接データを読み込みます（ブラウザに認証情報を渡さずに済みます）。初心者はず「URL から ID を取り出す → DB から1件取得 → 見つからなければ404、見つければ画面に描く」という大きな流れだけ追ってください。星表示やステータスの色分けなどの細かい処理はコード内のコメントで説明しています。

```

import { createClient } from "@lib/supabase/server"; // サーバー側で動く Supabase クライ
アントを作る関数を取り込む (@/ はプロジェクトルートを指すエイリアス)
import { notFound } from "next/navigation"; // 404 ページを表示するための Next.js 組み込み
関数
import Link from "next/link"; // クライアントサイド遷移ができる Next.js のリンクコンポーネント
(<a> よりも高速)
import DeleteButton from "@components/DeleteButton"; // この後作る削除ボタン (クライアント
コンポーネント)

// -----
// 型定義
// -----
// Next.js App Router のページコンポーネントは、
// params プロパティを受け取ります。
// [id] フォルダに配置されているので、params.id で
// URLの動的部分を取得できます。
// Next.js 15 以降は params が Promise でラップ
// されている点に注意してください。
// -----
type Props = {
  params: Promise<{ id: string }>; // params は「{ id: string } をいずれ返す Promise」型
};

// -----
// ステータス表示用のヘルパー関数
// -----
// データベースに保存されている英語のステータス値を
// 日本語の表示ラベルに変換します。
// 例: "unread" → "未読"
// -----
function getStatusLabel(status: string): string { // 引数 status は文字列、戻り値も文字列
  const statusMap: Record<string, string> = { // Record<キーの型, 値の型> は「キーと値の型
を指定したオブジェクト」を表す型
    unread: "未読", // DB の値 "unread" を「未読」に対応付け
    reading: "読書中", // DB の値 "reading" を「読書中」に対応付け
    finished: "読了", // DB の値 "finished" を「読了」に対応付け
  };
  return statusMap[status] || status; // 該当があれば日本語、無ければ元の値をそのまま返す (フ
ォールバック)
}

// -----
// ステータスに応じたバッジの色を返すヘルパー関数
// -----
// ステータスごとに異なる色のバッジを表示することで、
// ひと目で書籍の読書状態がわかるようにします。
// Tailwind CSS のクラス名を文字列として返します。
// -----
function getStatusColor(status: string): string {

```



```

switch (status) { // status の値で分岐
  case "unread": // 未読のとき
    return "bg-gray-100 text-gray-800"; // 背景はグレー、文字は濃いグレー
  case "reading": // 読書中のとき
    return "bg-blue-100 text-blue-800"; // 背景は薄い青、文字は濃い青
  case "finished": // 読了のとき
    return "bg-green-100 text-green-800"; // 背景は薄い緑、文字は濃い緑
  default: // 想定外の値が来たとき
    return "bg-gray-100 text-gray-800"; // フォールバックとしてグレー
}
}

// -----
// 評価を星マークで表示するヘルパー関数
// -----
// 数値の評価 (1~5) を視覚的な星マーク (★☆) に
// 変換して表示します。
// 例: 3 → "★★★☆☆"
// -----
function renderStars(rating: number | null): string { // rating は数値または null (評価なし)
  if (rating === null || rating === undefined) return "未評価"; // null/undefined のときは「未評価」と表示
  const filled = "★".repeat(rating); // 評価の数だけ「★」を繰り返す (例: 3 なら "★★★")
  const empty = "☆".repeat(5 - rating); // 残りの数だけ「☆」を繰り返す (例: 3 なら "☆☆")
  return filled + empty; // 「★★★」+「☆☆」=「★★★☆☆」を返す
}

// -----
// 書籍詳細ページコンポーネント (Server Component)
// -----
// このコンポーネントはサーバーサイドで実行されます。
// async 関数として定義することで、コンポーネント内で
// 直接 Supabase へのデータ取得を行えます。
// "use client" を書いていない=Server Component です。
// -----
export default async function BookDetailPage({ params }: Props) { // 分割代入で props から params を取り出し
  // -----
  // 1. params から書籍IDを取得
  // -----
  // Next.js 15 以降、params は Promise として渡される
  // ため、await で解決する必要があります。
  // ここで await を忘れると id が undefined になり、
  // データ取得が失敗します。
  // -----
  const { id } = await params; // Promise を解決して { id } を取り出す

```



```

// -----
// 2. Supabase クライアントの作成とデータ取得
// -----
// サーバーコンポーネント用の Supabase クライアントを
// 作成し、指定されたIDの書籍データを取得します。
// .single() を使うことで、配列ではなく単一の
// オブジェクトとしてデータを受け取ります。
// -----
const supabase = await createClient(); // Cookie 連携などの非同期処理があるため await が
必要

const { data: book, error } = await supabase // data を book という名前にリネームして受
け取る (分割代入のリネーム構文)
  .from("books") // 操作対象は books テーブル
  .select("*") // 全カラムを取得 ("id, title" のようにカラム名を絞ることも可能)
  .eq("id", id) // WHERE id = :id 条件 (URL から取った id と一致するレコード)
  .single(); // 結果を配列ではなく単一オブジェクトとして受け取る (0件や2件以上ならエラ
ー)

// -----
// 3. エラーハンドリング
// -----
// データが取得できなかった場合 (IDが存在しない、
// ネットワークエラーなど)、Next.js の notFound()
// 関数を呼び出して 404 ページを表示します。
//
// notFound() は例外をスローする関数なので、
// この行以降のコードは実行されません。
// try/catch で囲うとエラー扱いになってしまうので、
// 直接呼び出すのがポイントです。
// -----
if (error || !book) { // エラーがある、または book が空のとき
  notFound(); // 404 ページに即遷移 (以降のコードは実行されない)
}

// -----
// 4. 日付のフォーマット
// -----
// データベースに保存されている日付文字列を
// 日本語の表示形式に変換します。
// 例: "2024-01-15" → "2024年1月15日"
// -----
const formatDate = (dateString: string | null): string => { // アロー関数で日付フォーマ
ッタを定義
  if (!dateString) return "未設定"; // null や空文字なら「未設
定」を返す
  const date = new Date(dateString); // 文字列から Date オブ
ジェクトを生成
  return date.toLocaleDateString("ja-JP", { // 日本ロケールで整形
    "ja-JP" は日本語表記
  });
}

```

```

    year: "numeric", // 年は数字
    month: "long", // 月は「1月」のような長い形式
    day: "numeric", // 日は数字
  });
};

// -----
// 5. ページのレンダリング
// -----
// 詳細ページは以下のようなレイアウトで表示されます:
//
// カード形式のレイアウトで、書籍の全情報を
// 見やすく表示します。
// -----
return (
  <div className="max-w-2xl mx-auto py-8 px-4"> {/* 最大幅 2xl、左右中央寄せ、上下に余白 */}
    {/* -----
      戻るリンク
      一覧ページに戻るためのリンクです。
      ページ上部に配置して、ユーザーがすぐに
      一覧に戻れるようにします。
      ----- */}

    <Link
      href="/books" // 遷移先の URL
      className="inline-flex items-center text-sm text-blue-600 hover:text-blue-800
mb-6" // インラインフレックスで矢印とテキストを横並びに
    >
      <svg
        className="w-4 h-4 mr-1" // 16x16 のサイズ、右に少しマージン
        fill="none" // 塗りつぶしなし
        stroke="currentColor" // 線の色は親要素の文字色を引き継ぐ
        viewBox="0 0 24 24" // SVG の表示領域
      >
        <path
          strokeLinecap="round" // 線の端を丸く
          strokeLinejoin="round" // 線同士の結合点も丸く
          strokeWidth={2} // 線の太さ
          d="M15 19l-7-7 7-7" // ← の形を描画するパス
        />
      </svg>
      一覧に戻る
    </Link>

    {/* -----
      書籍詳細カード
      白背景のカードに影をつけて、情報を
      グループ化して表示します。
      ----- */}

    <div className="bg-white shadow-lg rounded-lg overflow-hidden"> {/* 白背景、強めの

```

```

影、角丸、はみ出し非表示 */}
    {/* -----
        ヘッダー部分
        書籍タイトルとステータスバッジを
        表示します。背景色をグラデーションに
        して視覚的に目立たせます。
    ----- */}

    <div className="bg-gradient-to-r from-blue-600 to-blue-800 px-6 py-8"> {/* 左か
    ら右へ青のグラデーション */}

        <div className="flex items-start justify-between"> {/* タイトルとバッジを左右に
        振り分ける */}

            <h1 className="text-2xl font-bold text-white">{book.title}</h1> {/*
            book.title を白文字で大きく表示 */}

            <span
                className={`inline-flex items-center px-3 py-1 rounded-full text-sm font-
                medium ${getStatusColor(book.status)}`} // テンプレートリテラルで動的にクラスを合成
            >
                {getStatusLabel(book.status)} {/* 日本語ラベルを表示 */}
            </span>
        </div>

        {book.author && ( /* book.author が真値 (空でない) のときだけ描画する短絡評価 */
            <p className="mt-2 text-blue-100 text-lg">{book.author}</p>
        )}
    </div>

    {/* -----
        詳細情報部分
        書籍の各項目を定義リスト風のレイアウトで
        表示します。ラベルと値を横並びにして、
        見やすく整理します。
    ----- */}

    <div className="px-6 py-6">
        <dl className="divide-y divide-gray-200"> {/* dl は定義リスト。子要素間に区切り線
        を入れる */}

            {/* 出版社 */}

            <div className="py-4 sm:grid sm:grid-cols-3 sm:gap-4"> {/* sm 以上で 3 カラ
            ムグリッド */}

                <dt className="text-sm font-medium text-gray-500">出版社</dt> {/* dt は定
                義の見出し (ラベル) */}

                <dd className="mt-1 text-sm text-gray-900 sm:col-span-2 sm:mt-0"> {/* dd
                は定義の本体 (値)。2カラム分使う */}

                    {book.publisher || "未設定"} {/* publisher が空なら「未設定」を表示 */}

                </dd>
            </div>

            {/* 出版日 */}

            <div className="py-4 sm:grid sm:grid-cols-3 sm:gap-4">

                <dt className="text-sm font-medium text-gray-500">出版日</dt>

                <dd className="mt-1 text-sm text-gray-900 sm:col-span-2 sm:mt-0">

                    {formatDate(book.published_date)} {/* 上で定義したフォーマットで整形 */}

```

```

        </dd>
    </div>

    {/* 評価 */}
    <div className="py-4 sm:grid sm:grid-cols-3 sm:gap-4">
        <dt className="text-sm font-medium text-gray-500">評価</dt>
        <dd className="mt-1 text-sm text-gray-900 sm:col-span-2 sm:mt-0">
            <span className="text-yellow-500 text-lg"> {/* 星は黄色、少し大きめ */}
                {renderStars(book.rating)} {/* 数値を ★★★☆☆ 形式に変換 */}
            </span>
        </dd>
    </div>

    {/* ステータス */}
    <div className="py-4 sm:grid sm:grid-cols-3 sm:gap-4">
        <dt className="text-sm font-medium text-gray-500">ステータス</dt>
        <dd className="mt-1 text-sm text-gray-900 sm:col-span-2 sm:mt-0">
            <span
                className={`inline-flex items-center px-2.5 py-0.5 rounded-full text-
xs font-medium ${getStatusColor(book.status)} `}
            >
                {getStatusLabel(book.status)}
            </span>
        </dd>
    </div>

    {/* メモ */}
    <div className="py-4 sm:grid sm:grid-cols-3 sm:gap-4">
        <dt className="text-sm font-medium text-gray-500">メモ</dt>
        <dd className="mt-1 text-sm text-gray-900 sm:col-span-2 sm:mt-0">
            {book.memo ? ( /* メモがあれば本文を、なければグレーで「メモはありません」を表
示する三項演算子 */
                <div className="bg-gray-50 rounded-md p-4 whitespace-pre-wrap"> {/*
whitespace-pre-wrap で改行を保持 */}
                    {book.memo}
                </div>
            ) : (
                <span className="text-gray-400">メモはありません</span>
            )}
        </dd>
    </div>

    {/* 登録日時 */}
    <div className="py-4 sm:grid sm:grid-cols-3 sm:gap-4">
        <dt className="text-sm font-medium text-gray-500">登録日時</dt>
        <dd className="mt-1 text-sm text-gray-900 sm:col-span-2 sm:mt-0">
            {formatDate(book.created_at)} {/* DB が自動で入れる作成日時 */}
        </dd>
    </div>
</dl>

```

```

</div>

{ /* -----
  アクションボタン部分
  編集と削除の操作ボタンを表示します。
  編集はリンク（青色）、削除はボタン
  （赤色）で表示して、操作を視覚的に
  区別します。
  ----- */}

<div className="bg-gray-50 px-6 py-4 flex items-center justify-end space-x-3">
{ /* 右寄せでボタンを横並び、要素間隔 3 */}
  <Link
    href={`\ /books/${book.id}/edit`} // テンプレートリテラルで動的に編集ページの URL
    を組み立て
    className="inline-flex items-center px-4 py-2 border border-transparent
text-sm font-medium rounded-md shadow-sm text-white bg-blue-600 hover:bg-blue-700
focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-blue-500"
    >
    <svg
      className="w-4 h-4 mr-2"
      fill="none"
      stroke="currentColor"
      viewBox="0 0 24 24"
    >
      <path
        strokeLinecap="round"
        strokeLinejoin="round"
        strokeWidth={2}
        d="M11 5H6a2 2 0 0-2 2v11a2 2 0 02 2h11a2 2 0 02-2v-5m-1.414-9.414a2
2 0 112.828 2.828L11.828 15H9v-2.828l8.586-8.586z" // 鉛筆（編集）アイコンのパス
      />
    </svg>
    編集する
  </Link>

  { /* -----
    DeleteButton はクライアントコンポーネント
    です。window.confirm による確認ダイアログ
    やクリックイベントの処理はブラウザ側で
    行う必要があるためです。
    サーバーコンポーネントから props を渡せる
    点に注目（id と title を渡している）。
    ----- */}

    <DeleteButton bookId={book.id} bookTitle={book.title} />
  </div>
</div>
</div>
);
}

```

このコードのポイント:

- `params` は Next.js 15 以降で `Promise` として渡されるため、`await params` で値を取得します。古いバージョンの Next.js では直接 `params.id` でアクセスできましたが、最新版ではこの非同期パターンが必要です。`await` を忘れると `id` が `undefined` になり、Supabase のクエリが失敗します。
- `notFound()` は `next/navigation` から import する関数で、呼び出すと即座に 404 ページが表示されます。内部的には例外をスローするため、`notFound()` 以降のコードは実行されません。ファイル単位で `not-found.tsx` を用意するとカスタマイズもできます。
- `.single()` メソッドは、クエリ結果が正確に1件であることを期待します。0件や2件以上の場合はエラーになります。「存在しないかも」を許容したいときは `.maybeSingle()` (0件のときは `data: null`、エラーにしない) を使うこともあります。
- サーバーコンポーネントなので `"use client"` ディレクティブは不要です。データ取得はすべてサーバーサイドで行われ、完成した HTML がクライアントに送信されます。ブラウザ側に Supabase の認証情報を晒さなくて済むメリットもあります。

詳細ページは上記のコード内のコメントで示した通り、カード形式のレイアウトで表示されます。完成イメージは以下の通りです:

書籍タイトル

著者	山田太郎
出版社	技術評論社
出版日	2024年1月15日
評価	★★★★☆
ステータス	読書中

メモ

とても良い本です。特に第3章が参考になりました。実務でも活用できる内容が多いです。

[編集する](#)[削除する](#)

ページ上部にはグラデーション背景のヘッダーがあり、書籍タイトル、著者名、ステータスバッジが表示されます。その下に出版社、出版日、評価（星マーク）、ステータス、メモ、登録日時が定義リスト風に並びます。ページ最下部にはグレー背景のフッターに「編集する」ボタン（青色）と「削除する」ボタン（赤色）が配置されます。

2. 編集機能

書籍情報を編集する機能を実装します。編集機能では、前章で作成した **BookForm** コンポーネントを再利用し、既存データを初期値としてフォームに表示します。

「編集」と冪等性: 編集 (UPDATE) は何度実行しても最終結果が同じになる操作 (冪等性がある) です。同じデータで 2 回更新ボタンを押しても、データベースの状態は同じになります。これは新規登録 (INSERT) と大きく異なる点で、INSERT は押すたびにレコードが増えてしまいます。

同時編集の競合について: 複数のユーザーが同じ書籍を同時に編集して保存すると、後から保存した方の内容で上書きされます (最後の書き込みが勝つ = Last Write Wins)。このチュートリアルの実装ではこの問題を意識せず単純に上書きしますが、より高度なアプリでは「更新日時を比較して古いデータの上書きを拒否する」「ロックを取る」といった対策が必要になります。

2-1. 編集処理のフロー

Phase 1: データ読み込み

- 1 ユーザー → ブラウザ: /books/[id]/edit にアクセス
- 2 編集ページ → Supabase: SELECT * FROM books WHERE id = :id
- 3 Supabase → 編集ページ → ブラウザ: フォーム (既存データ入り) をレンダリング

--- ユーザーがフォームを編集 ---

Phase 2: データ更新

- 4 ユーザー: 「更新」 ボタンをクリック
- 5 ブラウザ → Supabase: UPDATE books SET ... WHERE id = :id
- 6 更新完了: router.push("/books/[id]") → 詳細ページにリダイレクト

このフロー図が示すように、編集機能は2段階の処理で構成されています。まずページアクセス時にサーバーサイドで既存データを取得し、フォームの初期値として設定します (これを **編集前データのプリロード** と呼びます)。次にユーザーがフォームを編集して「更新」ボタンを押すと、クライアントサイドで Supabase の UPDATE クエリが実行され、完了後に詳細ページへリダイレクトされます。

楽観的 UI (Optimistic UI) について: より高速な UX を実現する手法として、サーバーの応答を待たずに「成功したであろう状態」を画面に先に反映してしまう手法があります。これを楽観的 UI と呼びます。本章の実装は逆で「サーバーの応答を待ってから反映する」シンプルな方式 (悲観的更新) です。初学者にはこちらが分かりやすいので、まずは応答を待つパターンで作ります。

2-2. BookForm の更新（編集対応）

前章で作成した `BookForm` コンポーネントを、新規登録と編集の両方に対応できるように更新します。主な変更点は以下の通りです。

- `initialData` prop: 編集時に既存データをフォームの初期値として渡す
- `isEdit` prop: 新規登録か編集かを区別する
- ボタンテキスト: 新規登録時は「登録する」、編集時は「更新する」
- 送信先の処理: 新規登録時は INSERT、編集時は UPDATE

defaultValue と value の違い: React の input には 2 通りの値の渡し方があります。 - `defaultValue` : 最初に 1 回だけ反映される（非制御コンポーネント）。ユーザーの入力は React の状態に同期されない。 - `value` : 常に渡された値が表示される（制御コンポーネント）。`onChange` で React の状態を更新しないと、入力できなくなる。本章のフォームは `value` + `onChange` の制御コンポーネント方式です。状態を React 側で完全に管理するので、検証や送信処理が書きやすくなります。

ファイル: `components/BookForm.tsx`

▼ このコードがやること（先に日本語で）：書籍の入力フォームを、「新規登録」と「編集」の両方で使い回せるように作り直します。ポイントは props で渡される `isEdit` というフラグで、これが `true` のときは Supabase の UPDATE（更新）を、`false` のときは INSERT（新規追加）を実行するよう処理を切り替えます。編集時は `initialData` で渡された既存データをフォームの初期値に入れておくので、最初から内容が埋まった状態で表示されます。初心者はまず「1つのフォームが2役をこなしている」「入力値は `useState` で React が管理している」という2点を押さえてください。各フィールドや送信処理の詳細はコード内のコメントにあります。

```
"use client"; // このファイルはブラウザ側で動くクライアントコンポーネント。useState/useRouter
// を使うために必須
```

```
import { useState } from "react"; // 状態管理フック
import { useRouter } from "next/navigation"; // ページ遷移フック (App Router 用)
import { createClient } from "@lib/supabase/client"; // ブラウザ向けの Supabase クライアント
```

```
// -----
```

```
// 型定義
```

```
// -----
```

```
// BookFormData: フォームで扱うデータの型
```

```
// initialState に渡す型でもあり、フォームの状態
```

```
// 管理にも使用します。
```

```
// 数値は null も許容 (評価が未入力の場合用)。
```

```
// -----
```

```
type BookFormData = {
  title: string;           // タイトル (必須)
  author: string;          // 著者
  publisher: string;        // 出版社
  published_date: string;   // 出版日 (YYYY-MM-DD 形式の文字列)
  rating: number | null;    // 評価 (1~5 または null)
  status: string;           // ステータス ("unread" / "reading" / "finished")
  memo: string;            // メモ
};
```

```
// -----
```

```
// Props の型定義
```

```
// -----
```

```
// initialState: 編集時に既存のデータを渡すための
```

```
// プロパティ。新規登録時は undefined。
```

```
// isEdit: 編集モードかどうかを示すフラグ。
```

```
// true の場合は UPDATE、false の場合は
```

```
// INSERT を実行します。
```

```
// bookId: 編集時に対象の書籍IDを渡すための
```

```
// プロパティ。UPDATE の WHERE 句で使用。
```

```
// 末尾の ? は「省略可能」を表す TypeScript の記法です。
```

```
// -----
```

```
type Props = {
  initialState?: BookFormData; // 省略可 (新規登録時は渡さない)
  isEdit?: boolean;           // 省略可 (デフォルト false)
  bookId?: string;            // 省略可 (編集時のみ必要)
};
```

```
// -----
```

```
// デフォルトの初期値
```

```
// -----
```

```
// 新規登録時に使用するフォームの初期値です。
```

```
// すべてのフィールドを空またはデフォルト値で
```

```

// 初期化します。
// -----
const defaultFormData: BookFormData = {
  title: "",           // 空文字
  author: "",          // 空文字
  publisher: "",       // 空文字
  published_date: "",  // 空文字 (date 入力欄の空状態)
  rating: null,        // 評価なし
  status: "unread",   // デフォルトは「未読」
  memo: "",           // 空文字
};

export default function BookForm({
  initialData,        // 編集時の初期データ
  isEdit = false,    // デフォルト値を指定 (渡されなければ false)
  bookId,            // 編集対象の ID
}: Props) {
  const router = useRouter(); // 遷移用のルーターを取得

  // -----
  // フォームの状態管理
  // -----
  // initialData が渡された場合 (編集モード) は
  // 既存データを初期値として使用します。
  // 渡されなかった場合 (新規登録モード) は
  // defaultFormData を初期値として使用します。
  // ?? は「左が null/undefined なら右を使う」演算子 (nullish coalescing) です。
  // -----
  const [formData, setFormData] = useState<BookFormData>(
    initialData ?? defaultFormData // initialData が undefined なら defaultFormData
  );
  const [isSubmitting, setIsSubmitting] = useState(false); // 送信中フラグ (二重送信防止)
  const [error, setError] = useState<string | null>(null); // エラーメッセージ (無ければ null)

  // -----
  // フォームフィールドの値変更ハンドラ
  // -----
  // 各入力フィールドの onChange イベントで呼ばれ、
  // フォームの状態を更新します。
  // name 属性をキーとして、対応するフィールドの
  // 値だけを更新します。
  // -----
  const handleChange = (
    e: React.ChangeEvent<
      HTMLInputElement | HTMLSelectElement | HTMLTextAreaElement // input / select /
      textarea のいずれかに対応
    >
  ) => {

```

```

const { name, value } = e.target; // イベントの発生元から name と value を取り出す
setFormData((prev) => ({          // 関数形式の setState (前の状態を引数に取れる)
  ...prev,                        // スプレッド構文で既存の値を全部展開 (他のフィールドは
そのまゝ)
  [name]: value,                  // 計算プロパティ名で name に対応するフィールドだけ上
書き
}));
});

// -----
// 評価の変更ハンドラ
// -----
// 評価は数値として扱うため、専用のハンドラを
// 用意します。空文字の場合は null を設定し、
// それ以外は数値に変換します。
// select の value は常に文字列で来るので、明示的に
// parseInt で数値化する必要があります。
// -----
const handleRatingChange = (e: React.ChangeEvent<HTMLSelectElement>) => {
  const value = e.target.value; // 選択された値 (文字列)
  setFormData((prev) => ({
    ...prev,
    rating: value === "" ? null : parseInt(value, 10), // 空なら null、そうでなければ
10 進数で整数化
  }));
});

// -----
// フォーム送信ハンドラ
// -----
// isEdit フラグに応じて INSERT または UPDATE を
// 実行します。
// async 関数として定義し、await で非同期処理の
// 完了を待ちます。
// -----
const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault(); // フォームのデフォルト送信 (ページリロード) をキャンセル
  setIsSubmitting(true); // 送信中状態に
  setError(null); // 過去のエラーをクリア

  // -----
  // バリデーション
  // -----
  // タイトルは必須項目です。空の場合はエラーを
  // 表示して処理を中断します。
  // trim() で前後の空白を除去してからチェックする
  // ことで「スペースだけ」の入力もエラー扱いに。
  // -----
  if (!formData.title.trim()) { // タイトルが空白のみ または 空文字の場合
    setError("タイトルは必須です。");
  }

```

```

    setIsSubmitting(false); // 送信中フラグを戻す
    return; // 処理を中断
}

try {
    const supabase = createClient(); // クライアント用 Supabase インスタンスを作成

    if (isEdit && bookId) { // 編集モード かつ ID がある場合
        // -----
        // 編集モード: UPDATE
        // -----
        // 既存の書籍データを更新します。
        // .eq("id", bookId) で対象のレコードを指定し、
        // フォームの内容で上書きします。
        // .eq() を省略すると全件更新されてしまうので
        // 絶対に忘れないこと。
        // -----
        const { error: updateError } = await supabase
            .from("books") // books テーブルに対して
            .update({ // UPDATE を発行
                title: formData.title.trim(), // 前後の空白を除去
                author: formData.author.trim() || null, // 空文字なら null (DB
// クリーンに保つ)
                publisher: formData.publisher.trim() || null,
                published_date: formData.published_date || null, // 空文字なら null
                rating: formData.rating, // null または数値
                status: formData.status,
                memo: formData.memo.trim() || null,
            })
            .eq("id", bookId); // WHERE id = bookId

        if (updateError) { // UPDATE 中にエラーが起きたら
            throw updateError; // catch ブロックに飛ばす
        }

        // -----
        // 更新成功: 詳細ページにリダイレクト
        // -----
        // 更新が完了したら、その書籍の詳細ページに
        // 遷移します。ユーザーは更新後の情報を
        // すぐに確認できます。
        // refresh() でサーバーコンポーネントのキャッシュ
        // も更新し、最新のデータが表示されるようにします。
        // -----
        router.push(`/books/${bookId}`); // 詳細ページに遷移
        router.refresh(); // サーバー側データを再取得
    } else {
        // -----
        // 新規登録モード: INSERT
        // -----

```

```

// 新しい書籍データを挿入します。
// .select() を付けることで、挿入されたデータ
// （自動生成されたIDを含む）を取得できます。
// -----
const { data, error: insertError } = await supabase
  .from("books")
  .insert({                                     // INSERT 文に相当
    title: formData.title.trim(),
    author: formData.author.trim() || null,
    publisher: formData.publisher.trim() || null,
    published_date: formData.published_date || null,
    rating: formData.rating,
    status: formData.status,
    memo: formData.memo.trim() || null,
  })
  .select()                                     // 挿入したレコードを返しても
  .single();                                   // 1 件だけ取得

if (insertError) {
  throw insertError;
}

// -----
// 登録成功: 一覧ページにリダイレクト
// -----
router.push("/books");
router.refresh();
}
} catch (err) {
  console.error("保存エラー:", err); // 開発者向けにコンソールへエラー出力
  setError(
    isEdit
      ? "書籍の更新に失敗しました。もう一度お試しください。" // 編集時のメッセージ
      : "書籍の登録に失敗しました。もう一度お試しください。" // 新規登録時のメッセージ
  );
} finally {
  setIsSubmitting(false); // 成功・失敗どちらでも、送信中フラグを戻す (finally で必ず実行)
}
};

// -----
// フォームのレンダリング
// -----
return (
  <form onSubmit={handleSubmit} className="space-y-6"> /* form の onSubmit で送信時に
handleSubmit が呼ばれる */
    { /* エラーメッセージ */ }
    {error && ( /* error が null でないときだけ描画 */

```

```

    <div className="bg-red-50 border-l-4 border-red-400 p-4 rounded"> { /* 赤系の警告
ボックス */}

    <div className="flex">
      <div className="flex-shrink-0">
        <svg
          className="h-5 w-5 text-red-400"
          viewBox="0 0 20 20"
          fill="currentColor"
        >
          <path
            fillRule="evenodd"
            d="M10 18a8 8 0 10-16 8 8 0 00 16 8z" data-bbox="10 18 18 20"/>
            1.414L8.586 10l-1.293 1.293a1 1 0 101.414 1.414L10 11.414l1.293 1.293a1 1 0 01.414-
            1.414L11.414 10l1.293-1.293a1 1 0 00-1.414-1.414L10 8.586 8.707 7.293z" // ×印アイコンの
            パス
            clipRule="evenodd"
          />
        </svg>
      </div>
      <div className="ml-3">
        <p className="text-sm text-red-700">{error}</p> { /* エラー文言を表示 */}
      </div>
    </div>
  )}

  { /* タイトル (必須) */}
  <div>
    <label
      htmlFor="title" // この label がどの input と紐づくか (id と一致させる)
      className="block text-sm font-medium text-gray-700"
    >
      タイトル <span className="text-red-500">*</span> { /* 赤いアスタリスクで必須を示す
*/}
    </label>
    <input
      type="text" // テキスト入力欄
      id="title" // label の htmlFor と一致
      name="title" // handleChange で参照される名前
      required // HTML 側の必須チェック (ブラウザがエラー表示)
      value={formData.title} // 制御コンポーネント方式: state の値を反映
      onChange={handleChange} // 入力たびに state を更新
      className="mt-1 block w-full rounded-md border-gray-300 shadow-sm
focus:border-blue-500 focus:ring-blue-500 sm:text-sm"
      placeholder="書籍のタイトルを入力" // 未入力時のヒント
    />
  </div>

  { /* 著者 */}
  <div>

```



```

<label
  htmlFor="author"
  className="block text-sm font-medium text-gray-700"
>
  著者
</label>
<input
  type="text"
  id="author"
  name="author" // name="author" → handleChange で
formData.author が更新される
  value={formData.author}
  onChange={handleChange}
  className="mt-1 block w-full rounded-md border-gray-300 shadow-sm
focus:border-blue-500 focus:ring-blue-500 sm:text-sm"
  placeholder="著者名を入力"
/>
</div>

{/* 出版社 */}
<div>
  <label
    htmlFor="publisher"
    className="block text-sm font-medium text-gray-700"
  >
    出版社
  </label>
  <input
    type="text"
    id="publisher"
    name="publisher"
    value={formData.publisher}
    onChange={handleChange}
    className="mt-1 block w-full rounded-md border-gray-300 shadow-sm
focus:border-blue-500 focus:ring-blue-500 sm:text-sm"
    placeholder="出版社名を入力"
  />
</div>

{/* 出版日 */}
<div>
  <label
    htmlFor="published_date"
    className="block text-sm font-medium text-gray-700"
  >
    出版日
  </label>
  <input
    type="date" // 日付ピッカー（YYYY-MM-DD 形式の文字列を扱う）
    id="published_date"

```

```

        name="published_date"
        value={formData.published_date}
        onChange={handleChange}
        className="mt-1 block w-full rounded-md border-gray-300 shadow-sm
focus:border-blue-500 focus:ring-blue-500 sm:text-sm"
      />
    </div>

    { /* 評価 */ }
    <div>
      <label
        htmlFor="rating"
        className="block text-sm font-medium text-gray-700"
      >
        評価
      </label>
      <select
        id="rating"
        name="rating"
        value={formData.rating ?? ""} // null のとき "" を渡す (select は string しか扱
えない)
        onChange={handleRatingChange} // 専用ハンドラで数値化
        className="mt-1 block w-full rounded-md border-gray-300 shadow-sm
focus:border-blue-500 focus:ring-blue-500 sm:text-sm"
      >
        <option value="">未評価</option> { /* 空文字を選択肢として用意 */ }
        <option value="1">★☆☆☆☆ (1)</option>
        <option value="2">★★☆☆☆ (2)</option>
        <option value="3">★★★☆☆ (3)</option>
        <option value="4">★★★★☆ (4)</option>
        <option value="5">★★★★★ (5)</option>
      </select>
    </div>

    { /* ステータス */ }
    <div>
      <label
        htmlFor="status"
        className="block text-sm font-medium text-gray-700"
      >
        ステータス
      </label>
      <select
        id="status"
        name="status"
        value={formData.status}
        onChange={handleChange} // ステータスは文字列のままなので共通ハンドラで OK
        className="mt-1 block w-full rounded-md border-gray-300 shadow-sm
focus:border-blue-500 focus:ring-blue-500 sm:text-sm"
      >

```

```

    <option value="unread">未読</option>
    <option value="reading">読書中</option>
    <option value="finished">読了</option>
  </select>
</div>

{/* ✕モ */}
<div>
  <label
    htmlFor="memo"
    className="block text-sm font-medium text-gray-700"
  >
    ✕モ
  </label>
  <textarea
    id="memo"
    name="memo"
    rows={4} // 表示行数の目安
    value={formData.memo}
    onChange={handleChange}
    className="mt-1 block w-full rounded-md border-gray-300 shadow-sm
focus:border-blue-500 focus:ring-blue-500 sm:text-sm"
    placeholder="読書✕モや感想を入力"
  />
</div>

{/* 送信ボタン */}
<div className="flex items-center justify-end space-x-3"> {/* ボタンを右寄せで横並
び */}
  <button
    type="button" // type="button" は「フォーム送信をしないボタン」
    onClick={() => router.back()} // クリック時に 1 つ前のページへ戻る
    className="inline-flex items-center px-4 py-2 border border-gray-300 text-sm
font-medium rounded-md text-gray-700 bg-white hover:bg-gray-50 focus:outline-none
focus:ring-2 focus:ring-offset-2 focus:ring-blue-500"
  >
    キャンセル
  </button>
  <button
    type="submit" // type="submit" でフォームの onSubmit を発火させ
る
    disabled={isSubmitting} // 送信中はクリック不可（二重送信防止）
    className="inline-flex items-center px-4 py-2 border border-transparent text-
sm font-medium rounded-md shadow-sm text-white bg-blue-600 hover:bg-blue-700
focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-blue-500
disabled:opacity-50 disabled:cursor-not-allowed"
  >
    {isSubmitting // 送信中なら... */
      ? isEdit // かつ編集モードなら */
        ? "更新中..." // 「更新中...」 */
    }
  </button>
</div>

```

```

      : "登録中..."          /* それ以外は「登録中...」 */
      : isEdit                 /* 送信中でなくて編集モードなら */
      ? "更新する"           /* 「更新する」 */
      : "登録する"}          /* 新規登録なら「登録する」 */
    </button>
  </div>
</form>
);
}

```

BookForm の主な変更点の解説:

1. **initialData prop**: 編集時に既存のデータをフォームの初期値として渡します。**useState** の初期値で **initialData ?? defaultFormData** と記述することで、initialData が渡された場合はそのデータを、渡されなかった場合はデフォルト値を使用します。これが「編集前データのプリロード」の中核です。
2. **isEdit prop**: このフラグが **true** の場合、フォーム送信時に **INSERT** ではなく **UPDATE** を実行します。デフォルト値は **false**（新規登録モード）です。
3. **bookId prop**: 編集時に UPDATE の対象を特定するために必要です。**.eq("id", bookId)** で特定のレコードだけを更新します。これを忘れると「全件更新」という事故につながるので注意してください。
4. ボタンテキストの切り替え: **isEdit** フラグに応じて、ボタンのテキストが「登録する」と「更新する」で切り替わります。送信中の表示も「登録中...」と「更新中...」で異なります。
5. リダイレクト先の違い: 新規登録後は一覧ページ（**/books**）へ、編集後は詳細ページ（**/books/\${bookId}**）へリダイレクトします。

type="submit" と type="button" の違い: **<form>** 内のボタンは、**type** 属性を指定しないと既定で **submit** 扱いになり、クリックするとフォームが送信されてしまいます。「キャンセル」のように送信を伴わないボタンには必ず **type="button"** を明示してください。逆に「更新する／登録する」ボタンは送信したいので **type="submit"** を指定し、フォームの **onSubmit** ハンドラを発火させます。

編集フォームにアクセスすると、各入力フィールドに既存のデータが入った状態で表示されます。ユーザーは変更したい項目だけを修正して「更新する」ボタンを押せば、データベースが更新されます。

2-3. 編集ページの作成

app/books/[id]/edit/ ディレクトリを作成し、**page.tsx** を作成します。このページはサーバーコンポーネントとして動作し、既存の書籍データを取得して **BookForm** に渡す役割を持ちます。

ファイル: **app/books/[id]/edit/page.tsx**

▼このコードがやること（先に日本語で）：編集ページの本体です。URL の `[id]` から書籍を1件取得し、その既存データを `initialData` として `BookForm` に渡すのが役割です。このページ自体はサーバー側でデータを読むだけの Server Component で、実際の入力や更新処理は受け取った側の `BookForm`（クライアント側）が担当します。初心者は「重いデータ取得はサーバーで行い、対話的なフォームはクライアントに任せる」という役割分担に注目してください。`null` を空文字に変換している理由などはコード内のコメントで説明しています。

```

import { createClient } from "@lib/supabase/server"; // サーバー用 Supabase クライアント
import { notFound } from "next/navigation"; // 404 表示用
import Link from "next/link"; // 戻るリンク用
import BookForm from "@components/BookForm"; // 上で更新した共通フォーム

// -----
// 型定義
// -----
// 動的セグメント [id] の値を Promise として受け取る
// (Next.js 15 以降の仕様)。
// -----
type Props = {
  params: Promise<{ id: string }>;
};

// -----
// 編集ページコンポーネント (Server Component)
// -----
// このコンポーネントは以下の処理を行います:
// 1. URL のパラメータから書籍IDを取得
// 2. Supabase から既存の書籍データを取得
// 3. データが存在しなければ 404 を表示
// 4. BookForm コンポーネントに既存データを渡して
// レンダリング
// -----
export default async function BookEditPage({ params }: Props) {
  // -----
  // 1. params から書籍IDを取得
  // -----
  // await を忘れると id が undefined になり、
  // データ取得時にエラーになります。
  // -----
  const { id } = await params;

  // -----
  // 2. Supabase から既存データを取得
  // -----
  const supabase = await createClient(); // クライアントを準備

  const { data: book, error } = await supabase
    .from("books")
    .select("*")
    .eq("id", id) // URL の id と一致するレコード
    .single(); // 単一レコードとして取得

  // -----
  // 3. エラーハンドリング
  // -----
  // 存在しない ID にアクセスされた場合や、

```

```

// 何らかのエラーが起きた場合は 404 を表示します。
// -----
if (error || !book) {
  notFound();
}

// -----
// 4. BookForm に渡す初期データを整形
// -----
// データベースから取得したデータをフォームの型に
// 合わせて整形します。null の値は空文字に変換し、
// フォームの入力フィールドで正しく表示されるように
// します。
//
// ?? は左が null/undefined のときだけ右の値を使う
// 演算子です (nullish coalescing)。
// -----
const initialData = {
  title: book.title ?? "", // null なら空文字に変換
  author: book.author ?? "",
  publisher: book.publisher ?? "",
  published_date: book.published_date ?? "",
  rating: book.rating ?? null, // null はそのまま
  status: book.status ?? "unread", // status は null だと困るのでデフォルトを設定
  memo: book.memo ?? "",
};

// -----
// 5. レンダリング
// -----
return (
  <div className="max-w-2xl mx-auto py-8 px-4">
    {/* 戻るリンク */}
    <Link
      href={`\books/${id}`} // 編集元の詳細ページへ
      className="inline-flex items-center text-sm text-blue-600 hover:text-blue-800
mb-6"
    >
      <svg
        className="w-4 h-4 mr-1"
        fill="none"
        stroke="currentColor"
        viewBox="0 0 24 24"
      >
        <path
          strokeLinecap="round"
          strokeLinejoin="round"
          strokeWidth={2}
          d="M15 19l-7-7 7-7" // 左向き矢印

```

```

    />
  </svg>
  詳細に戻る
</Link>

  { /* ページタイトル */ }
  <div className="mb-8">
    <h1 className="text-2xl font-bold text-gray-900">書籍を編集</h1>
    <p className="mt-1 text-sm text-gray-600">
      「{book.title}」の情報を編集します。 { /* 既存タイトルを表示して、編集対象を明示 */ }
    </p>
  </div>

  { /* -----
    BookForm コンポーネント
    initialData: 既存の書籍データ
    isEdit: true (編集モード)
    bookId: 対象の書籍ID
    サーバーコンポーネントからクライアント
    コンポーネントへ props を渡す例。
    ----- */ }
  <div className="bg-white shadow rounded-lg p-6">
    <BookForm initialData={initialData} isEdit={true} bookId={id} />
  </div>
</div>
);
}

```

このコードのポイント:

- 編集ページもサーバーコンポーネントです。データ取得はサーバーサイドで行い、取得したデータを **BookForm** (クライアントコンポーネント) に props として渡します。「重い処理はサーバー」「対話的な処理はクライアント」という Next.js App Router の典型的な役割分担です。
- **initialData** の整形では、データベースの **null** 値を空文字に変換しています。これにより、フォームの入力フィールドが React の controlled component として正しく動作します (**null** を value に渡すと uncontrolled component になる) 。
- 戻るリンクは詳細ページ (**/books/\${id}**) を指しています。一覧ではなく、編集元の詳細ページに戻る方がユーザー体験として自然です。

Server Action という別パターン: 本章ではクライアントコンポーネント側で Supabase を直接呼び出していますが、Next.js には「Server Action」という別の更新手段もあります。「**use server**」ディレクティブを付けた関数をフォームに直接渡し、サーバー側で更新→ **revalidatePath** でキャッシュ破棄→ **redirect** で遷移、という流れにできます。今回の教材では学習コストを抑えるためクライアント直書きを採用していますが、認証情報を守りたい場合や複雑なバリデーションをサーバーで行いたい場合は Server Action の方が向いています。

revalidatePath("/books") を呼ぶと、その URL に紐づくサーバーコンポーネントのキャッシュが破棄され、次のアクセスで最新データが取得されます。

3. 削除機能

書籍を削除する機能を実装します。削除は取り消しができない操作なので、**確認ダイアログ**を表示してユーザーの意思を再確認してから実行します。

3-1. 削除処理のフロー

1

ユーザー → DeleteButton

「削除する」ボタンをクリック

2

DeleteButton → 確認ダイアログ

window.confirm() → 「本当に削除しますか？」

キャンセルを選択した場合

✕ confirm() → false を返す
何もしない（処理を中断）

OKを選択した場合

- 3 ローディング状態に変更
- 4 Supabase: DELETE FROM books WHERE id = :id
- 5 削除完了 → router.push("/books")
- 6 一覧ページにリダイレクト

削除ボタンを押すと、まず **window.confirm()** による確認ダイアログが表示されます。「本当に "書籍タイトル" を削除しますか？この操作は取り消せません。」というメッセージが表示され、ユーザーが「OK」を選択した場合のみ削除処理が実行されます。「キャンセル」を選択した場合は何も起こりません。

削除確認モダルの選択肢: **window.confirm()** はブラウザ標準のシンプルな確認ダイアログです。お手軽ですが見た目の自由度が低いため、本格的なプロダクトでは React の状態でモダル（自前の確認画面）を出すパターンも一般的です。今回はシンプルさを優先して **window.confirm** を採用しています。

3-2. DeleteButton コンポーネントの作成

削除ボタンはクライアントサイドのインタラクション（クリックイベント、確認ダイアログ）を必要とするため、`"use client"` ディレクティブを付けたクライアントコンポーネントとして作成します。

ファイル: `components/DeleteButton.tsx`

▼ このコードがやること（先に日本語で）：「削除する」ボタンを作ります。クリックするとまず `window.confirm()` でブラウザ標準の確認ダイアログを出し、ユーザーが「OK」を押したときだけ Supabase の DELETE（削除）を実行します。確認・クリック処理はブラウザ側でしか動かないため、先頭に `"use client"` を付けたクライアントコンポーネントにしています。初心者は「①確認 → ②OKなら削除 → ③一覧へ戻る」という流れと、処理中はボタンを無効化して二重削除を防いでいる点を押さえてください。詳細はコード内のコメントにあります。

```

"use client"; // クライアントコンポーネント宣言:useState や onClick を使うために必須

import { useState } from "react"; // 削除中フラグ管理用
import { useRouter } from "next/navigation"; // 削除後の遷移用
import { createClient } from "@lib/supabase/client"; // ブラウザ向け Supabase クライアント

// -----
// Props の型定義
// -----
// bookId: 削除対象の書籍ID。
// Supabase の DELETE クエリの WHERE 句で
// 使用します。
// bookTitle: 確認ダイアログに表示する書籍タイトル。
// ユーザーが削除対象を確認できるように
// するために使用します。
// -----
type Props = {
  bookId: string;
  bookTitle: string;
};

export default function DeleteButton({ bookId, bookTitle }: Props) {
  const router = useRouter(); // 削除後に一覧へ遷移するために使用

  // -----
  // 状態管理
  // -----
  // isDeleting: 削除処理中かどうかを管理します。
  // true の間はボタンを無効化し、
  // テキストを「削除中...」に変更して、
  // 二重送信を防ぎます。
  // -----
  const [isDeleting, setIsDeleting] = useState(false);

  // -----
  // 削除処理ハンドラ
  // -----
  // async 関数なので、内部で await を使えます。
  // -----
  const handleDelete = async () => {
    // -----
    // 1. 確認ダイアログの表示
    // -----
    // window.confirm() はブラウザ標準の確認ダイアログ
    // を表示します。ユーザーが「OK」をクリックすると
    // true、「キャンセル」をクリックすると false を
    // 返します。
    //
  }

```

```

// 確認ダイアログのメッセージには書籍タイトルを
// 含め、削除対象が明確になるようにします。
// \n で改行を入れています。
// -----
const confirmed = window.confirm(
  `本当に「${bookTitle}」を削除しますか？\nこの操作は取り消せません。`
);

// -----
// キャンセルされた場合は何もしない
// -----
// 早期 return パターン。深いネストを避けて
// 読みやすくする定番の書き方です。
// -----
if (!confirmed) {
  return;
}

// -----
// 2. 削除処理の実行
// -----
setIsDeleting(true); // ローディング状態に切り替え

try {
  const supabase = createClient(); // Supabase クライアントを準備

  // -----
  // Supabase DELETE クエリ
  // -----
  // .from("books") で books テーブルを指定し、
  // .delete() で削除操作を行います。
  // .eq("id", bookId) で削除対象を書籍IDで
  // 絞り込みます。
  //
  // ※ .eq() を付けずに .delete() だけを実行すると
  // テーブルの全データが削除されるので、
  // 必ず WHERE 条件を指定してください。
  // (※実際には RLS や Supabase の安全装置で
  // そう簡単には全削除されませんが、習慣として
  // 必ず付けるクセを付けましょう)
  // -----
  const { error } = await supabase
    .from("books")
    .delete() // DELETE 文に相当
    .eq("id", bookId); // WHERE id = bookId

  if (error) {
    throw error; // catch にエラーを渡す
  }
}

```

```

// -----
// 3. 削除成功: 一覧ページにリダイレクト
// -----
// 削除した書籍の詳細ページはもう存在しないため、
// 一覧ページに遷移します。
// router.refresh() でサーバーコンポーネントの
// データを再取得し、削除された書籍が一覧から
// 消えていることを確認できるようにします。
// -----

router.push("/books"); // 一覧ページへ
router.refresh();      // サーバー側データを再取得
} catch (err) {
  console.error("削除エラー:", err);
  // -----
  // エラーが発生した場合はアラートで通知
  // -----
  // alert はブラウザ標準のメッセージダイアログ。
  // 簡易的なエラー表示として使っています。
  // -----
  alert("書籍の削除に失敗しました。もう一度お試しください。");
  setIsDeleting(false); // 状態を戻してリトライ可能に
}
};

// -----
// ボタンのレンダリング
// -----
// 削除ボタンは赤色で表示し、危険な操作であることを
// 視覚的に示します。
// 削除処理中はボタンを無効化し、テキストを
// 「削除中…」に変更して、処理中であることを
// ユーザーに伝えます。
// -----

return (
  <button
    onClick={handleDelete} // クリックで
削除ハンドラを呼ぶ
    disabled={isDeleting} // 削除中は
クリック不可
    className="inline-flex items-center px-4 py-2 border border-transparent text-sm
font-medium rounded-md shadow-sm text-white bg-red-600 hover:bg-red-700 focus:outline-
none focus:ring-2 focus:ring-offset-2 focus:ring-red-500 disabled:opacity-50
disabled:cursor-not-allowed"
  >
    {isDeleting ? ( /* 削除中はスピナー+「削除中…」を表示 */
      <>
        {/* -----
ローディングスピナー
削除処理中に表示される回転アニメーション
です。ユーザーに処理中であることを

```

転ア二メ

}

このコードのポイント:

- `"use client"` ディレクティブが必須です。 `window.confirm()` や `onClick` イベントハンドラはブラウザ側でしか動作しないためです。サーバー側には `window` オブジェクト自体が存在しません。
- `isDeleting` 状態によるローディング管理で、二重クリックによる二重削除を防いでいます。ボタンの `disabled` 属性と組み合わせることで、処理中は追加のクリックを受け付けません。なお DELETE は幂等（何度実行しても結果が同じ）なので、仮に 2 回送られてもデータが壊れることはありませんが、不要なネットワーク負荷を避ける意味でガードします。
- 削除後は `router.push("/books")` で一覧ページに遷移し、 `router.refresh()` でサーバーコンポーネントのキャッシュを更新します。これにより、一覧ページで最新のデータが表示されます。
- エラー時は `alert()` でユーザーに通知し、 `isDeleting` を `false` に戻してリトライ可能にしています。

このボタンは独立した `<button>` であり、フォームの `type="submit"` ではありません。もしフォーム内に置く場合は `type="button"` を明示しないと、親フォームの送信が発火してしまうので注意してください。今回は親が `<form>` ではなくただの `<div>` なのでこの問題は起きません。

4. 検索・フィルタ機能（発展）

一覧ページに検索バーとステータスフィルタを追加し、大量の書籍データの中から目的の本を素早く見つけられるようにします。

4-1. 検索バーコンポーネント

タイトルや著者名で書籍を検索できるコンポーネントを作成します。URL の検索パラメータ（ `searchParams` ）を活用して、検索状態をURLに反映させます。これにより、検索結果のURLを共有したり、ブラウザの戻るボタンで検索前の状態に戻ったりできます。

ファイル: `components/SearchBar.tsx`

▼ このコードがやること（先に日本語で）：タイトル・著者名で書籍を絞り込む検索バーと、ステータスで絞り込むフィルタを作ります。検索条件は state ではなく URL の `?q=...&status=...`（検索パラメータ）に反映させるのがポイントで、こうすると検索結果のURLを共有したり、戻るボタンで前の状態に戻したりできます。さらに「キーを打った後に検索せず、入力が止まって300ミリ秒後にまとめて検索する」デバウンスという工夫も入れています。初心者は「入力 → URL を書き換える → サーバーがその条件で再取得する」という流れだけ追えば十分で、`useEffect` やデバウンスの細部はコメントで補足しています。

```

"use client"; // 入力イベントや setState を扱うのでクライアントコンポーネント

import { useRouter, useSearchParams, usePathname } from "next/navigation"; // ルーター、
URL クエリ、パス取得用フック
import { useState, useEffect, useCallback } from "react"; // 状態管理・副作用・関数メモ化

// -----
// 検索バーコンポーネント
// -----
// このコンポーネントは以下の機能を提供します:
// 1. テキスト入力による検索
// 2. ステータスによるフィルタリング
// 3. URLの検索パラメータとの同期
// 4. デバウンス処理（入力のたびにクエリを
//    発行しないように、一定時間待ってから検索）
// -----
export default function SearchBar() {
  const router = useRouter(); // URL を変更するため
  const searchParams = useSearchParams(); // 現在の ?q=... を読むため
  const pathname = usePathname(); // 現在のパス（例: /books）を取得

  // -----
  // URL の検索パラメータから初期値を取得
  // -----
  // ページをリロードしたり、検索結果のURLに直接
  // アクセスしたりした場合でも、検索状態が保持
  // されるようにします。
  // -----
  const [searchQuery, setSearchQuery] = useState(
    searchParams.get("q") ?? "" // ?q= の値、無ければ空文字
  );
  const [statusFilter, setStatusFilter] = useState(
    searchParams.get("status") ?? "" // ?status= の値、無ければ空文字
  );

  // -----
  // URL の検索パラメータを更新する関数
  // -----
  // 検索クエリやフィルタが変更されたときに、
  // URL の検索パラメータを更新してページを
  // 再レンダリングします。
  // useCallback で関数をメモ化することで、
  // 依存配列が変わらない限り同じ関数インスタンスを
  // 使い回します。
  // -----
  const updateSearchParams = useCallback(
    (query: string, status: string) => {
      const params = new URLSearchParams(searchParams.toString()); // 現在のクエリを元に
      コピーを作成
    }
  );

```



```

// -----
// 検索クエリの設定
// -----
// 値が空の場合はパラメータを削除し、
// URL をクリーンに保ちます。
// 例: /books?q=React&status=reading
//      検索クエリが空なら /books?status=reading
// -----
if (query.trim()) { // 空白以外の文字が含まれていれば
  params.set("q", query.trim()); // ?q= を設定
} else {
  params.delete("q"); // 空なら ?q= を削除
}

// ステータスフィルタの設定
if (status) {
  params.set("status", status);
} else {
  params.delete("status");
}

// -----
// URL を更新
// -----
// router.push() を使って URL を更新すると、
// Next.js が自動的にサーバーコンポーネントを
// 再レンダリングし、新しい searchParams で
// データを再取得します。
// -----
const queryString = params.toString(); //
"q=React&status=reading" など
const newUrl = queryString ? `${pathname}?${queryString}` : pathname; // クエリが
あれば付加、なければパスだけ
router.push(newUrl); // URL を更新
},
[router, pathname, searchParams] // これらが変わったら関数を作り直す
);

// -----
// デバウンス処理
// -----
// ユーザーがキーボードを打った時に検索クエリを
// 発行すると、不必要なリクエストが大量に発生
// します。デバウンスを使って、ユーザーが入力を
// 止めてから 300ms 後に検索を実行します。
// useEffect は「描画後に副作用を実行する」フック。
// -----
useEffect(() => {
  const timer = setTimeout(() => { // 300ms 後に実行する予約

```

```

    updateSearchParams(searchQuery, statusFilter);
  }, 300);

  // -----
  // クリーンアップ関数
  // -----
  // 次の入力が行われた場合、前のタイマーを
  // キャンセルします。これにより、最後の入力
  // から 300ms 後にのみ検索が実行されます。
  // -----
  return () => clearTimeout(timer); // 副作用の片付け
}, [searchQuery, statusFilter, updateSearchParams]); // これらが変わるたびに発火

// -----
// 検索条件クリア
// -----
// 入力欄を空にして、URL も検索条件無しに戻します。
// -----
const handleClear = () => {
  setSearchQuery(""); // 検索クエリをクリア
  setStatusFilter(""); // フィルタをクリア
  router.push(pathname); // ?xxx を全部外す
};

// -----
// レンダリング
// -----
return (
  <div className="bg-white shadow rounded-lg p-4 mb-6">
    <div className="flex flex-col sm:flex-row gap-4"> {/* モバイルは縦、PC は横並び */}
      {/* 検索入力フィールド */}
      <div className="flex-1">
        <label htmlFor="search" className="sr-only"> {/* sr-only は視覚的には非表示、スクリーンリーダー用 */}
          検索
        </label>
        <div className="relative">
          {/* 虫眼鏡アイコン */}
          <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-none"> {/* クリックを通過させる */}
            <svg
              className="h-5 w-5 text-gray-400"
              fill="none"
              stroke="currentColor"
              viewBox="0 0 24 24"
            >
              <path
                strokeLinecap="round"
                strokeLinejoin="round"
                strokeWidth={2}

```

```

        d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z" // 虫眼鏡のパス
      />
    </svg>
  </div>
  <input
    type="text"
    id="search"
    value={searchQuery} // state を反映（制御コン
ポーネント）
    onChange={(e) => setSearchQuery(e.target.value)} // 入力のたびに state を
更新
    placeholder="タイトルまたは著者名で検索..."
    className="block w-full pl-10 pr-3 py-2 border border-gray-300 rounded-md
leading-5 bg-white placeholder-gray-500 focus:outline-none focus:placeholder-gray-400
focus:ring-1 focus:ring-blue-500 focus:border-blue-500 sm:text-sm"
  />
</div>
</div>

{/* ステータスフィルタ */}
<div className="sm:w-48">
  <label htmlFor="status-filter" className="sr-only">
    ステータスフィルタ
  </label>
  <select
    id="status-filter"
    value={statusFilter}
    onChange={(e) => setStatusFilter(e.target.value)}
    className="block w-full py-2 px-3 border border-gray-300 bg-white rounded-
md shadow-sm focus:outline-none focus:ring-blue-500 focus:border-blue-500 sm:text-sm"
  >
    <option value="">すべてのステータス</option>
    <option value="unread">未読</option>
    <option value="reading">読書中</option>
    <option value="finished">読了</option>
  </select>
</div>

{/* クリアボタン */}
{({searchQuery || statusFilter}) && ( /* どちらか入っているときだけ表示 */
  <button
    onClick={handleClear}
    className="inline-flex items-center px-3 py-2 border border-gray-300 text-
sm font-medium rounded-md text-gray-700 bg-white hover:bg-gray-50 focus:outline-none
focus:ring-2 focus:ring-offset-2 focus:ring-blue-500"
  >
    <svg
      className="w-4 h-4 mr-1"
      fill="none"
      stroke="currentColor"

```

```

        viewBox="0 0 24 24"
      >
        <path
          strokeLinecap="round"
          strokeLinejoin="round"
          strokeWidth={2}
          d="M6 18L18 6M6 6L12 12" // × アイコン
        />
      </svg>
      クリア
    </button>
  )}
</div>
</div>
);
}

```

4-2. 一覧ページの更新（検索・フィルタ対応）

検索バーコンポーネントを一覧ページに組み込み、Supabase のクエリを検索条件に対応させます。

ファイル: `app/books/page.tsx`（更新版）

▼ このコードがやること（先に日本語で）：前章で作った一覧ページを、検索・フィルタ・ソートに対応させた更新版です。URL の検索パラメータ（`q` / `status` / `sort` / `order`）を読み取り、その条件に合わせて Supabase へのクエリを少しずつ組み立てていきます。`ilike` は大文字小文字を区別しない部分一致検索、`.or()` は「タイトル または 著者名」のどちらかにマッチさせる指定です。初心者は「URL の条件を見て → クエリに条件を足していき → 最後にまとめて実行する」という組み立て方に注目してください。各条件の詳細はコード内のコメントで説明しています。

```

import { createClient } from "@lib/supabase/server"; // サーバー用 Supabase クライアント
import Link from "next/link"; // 詳細ページ等への遷移
import SearchBar from "@components/SearchBar"; // 検索バー (クライアントコンポーネント)
import SortSelect from "@components/SortSelect"; // ソート用セレクト (クライアントコンポーネント)

// -----
// 型定義
// -----
// Next.js App Router のページコンポーネントは、
// searchParams プロパティでURLの検索パラメータを
// 受け取ることができます。
// Next.js 15 以降は Promise でラップされる点に注意。
// -----
type Props = {
  searchParams: Promise<{ // URL の ? 以降のパラメータ
    q?: string; // 検索クエリ (省略可)
    status?: string; // ステータスフィルタ
    sort?: string; // ソート対象カラム
    order?: string; // 昇順/降順
  }>;
};

// -----
// ステータス表示用のヘルパー関数
// -----
// 詳細ページで使ったものと同じ。
// 共通化したい場合は別ファイルに抽出するとよい。
// -----
function getStatusLabel(status: string): string {
  const statusMap: Record<string, string> = {
    unread: "未読",
    reading: "読書中",
    finished: "読了",
  };
  return statusMap[status] || status;
}

function getStatusColor(status: string): string {
  switch (status) {
    case "unread":
      return "bg-gray-100 text-gray-800";
    case "reading":
      return "bg-blue-100 text-blue-800";
    case "finished":
      return "bg-green-100 text-green-800";
    default:
      return "bg-gray-100 text-gray-800";
  }
}

```

```

}

// -----
// 評価を星マークで表示する関数
// -----
function renderStars(rating: number | null): string {
  if (rating === null || rating === undefined) return "-"; // 未評価のときはハイフン
  return "★".repeat(rating) + "☆".repeat(5 - rating);
}

// -----
// 書籍一覧ページ (Server Component)
// -----
export default async function BooksPage({ searchParams }: Props) {
  const { q, status, sort, order } = await searchParams; // Promise を解決して各パラメータを取り出し
  const supabase = await createClient(); // サーバー用クライアント

  // -----
  // Supabase クエリの構築
  // -----
  // 基本のクエリを作成し、検索条件やフィルタ条件が
  // ある場合は動的にクエリを組み立てます。
  // let で宣言して、後から条件を追加していく形にします。
  // -----
  let query = supabase.from("books").select("*"); // ベースのクエリ (全件取得)

  // -----
  // テキスト検索
  // -----
  // q パラメータが指定されている場合、タイトルまたは
  // 著者名で部分一致検索を行います。
  //
  // Supabase の ilike は大文字小文字を区別しない
  // LIKE 検索です。% はワイルドカードで、
  // %検索語% は「検索語を含む」という意味になります。
  //
  // .or() を使って、タイトルまたは著者名のいずれかに
  // マッチする条件を設定します。
  // -----
  if (q) {
    query = query.or(`title.ilike.${q},author.ilike.${q}`); // title OR author で部
    分一致
  }

  // -----
  // ステータスフィルタ
  // -----
  // status パラメータが指定されている場合、
  // そのステータスの書籍のみを取得します。

```

```

// -----
if (status) {
  query = query.eq("status", status); // WHERE status = :status
}

// -----
// ソート
// -----
// sort パラメータでソート対象のカラムを、
// order パラメータでソート順（昇順/降順）を
// 指定します。デフォルトは作成日の降順です。
// -----
const sortColumn = sort || "created_at"; // 指定が無ければ created_at
const ascending = order === "asc"; // "asc" のときだけ昇順
query = query.order(sortColumn, { ascending }); // ORDER BY :sortColumn :order

// -----
// クエリ実行
// -----
// ここで初めてサーバーへリクエストが飛ぶ。
// -----
const { data: books, error } = await query;

if (error) {
  console.error("書籍の取得に失敗:", error);
}

// -----
// レンダリング
// -----
return (
  <div className="max-w-4xl mx-auto py-8 px-4">
    { /* ページヘッダー */ }
    <div className="flex items-center justify-between mb-6">
      <h1 className="text-2xl font-bold text-gray-900">書籍一覧</h1>
      <Link
        href="/books/new"
        className="inline-flex items-center px-4 py-2 border border-transparent text-sm font-medium rounded-md shadow-sm text-white bg-blue-600 hover:bg-blue-700"
      >
        <svg
          className="w-4 h-4 mr-2"
          fill="none"
          stroke="currentColor"
          viewBox="0 0 24 24"
        >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            strokeWidth={2}

```

```

        d="M12 4v16m8-8H4" // + アイコン
    />
</svg>
    新規登録
</Link>
</div>

{ /* 検索バーとソート */ }
<SearchBar />
<SortSelect />

{ /* 検索結果の件数表示 */ }
<p className="text-sm text-gray-500 mb-4">
    {books ? `${books.length}件の書籍が見つかりました` : "読み込み中..." }
    {q && (
        <span className="ml-2">
            (検索: 「{q}」)
        </span>
    )}
    {status && (
        <span className="ml-2">
            (フィルタ: {getStatusLabel(status)})
        </span>
    )}
</p>

{ /* 書籍一覧 */ }
{!books || books.length === 0 ? ( /* データが無いとき */
    <div className="text-center py-12 bg-white rounded-lg shadow">
        <svg
            className="mx-auto h-12 w-12 text-gray-400"
            fill="none"
            stroke="currentColor"
            viewBox="0 0 24 24"
        >
            <path
                strokeLinecap="round"
                strokeLinejoin="round"
                strokeWidth={2}
                d="M12 6.253v13m0-13C10.832 5.477 9.246 5 7.5 5S4.168 5.477 3
6.253v13C4.168 18.477 5.754 18 7.5 18s3.332.477 4.5 1.253m0-13C13.168 5.477 14.754 5
16.5 5c1.747 0 3.332.477 4.5 1.253v13C19.832 18.477 18.247 18 16.5 18c-1.746 0-
3.332.477-4.5 1.253" // 開いた本のアイコン
            />
        </svg>
        <h3 className="mt-2 text-sm font-medium text-gray-900">
            書籍が見つかりません
        </h3>
        <p className="mt-1 text-sm text-gray-500">
            {q || status

```



```

      ? "検索条件を変更してお試しください。"
      : "新しい書籍を登録してみましょう。"}
    </p>
    {!q && !status && ( /* 検索条件無しで 0 件のときだけ「最初の書籍を登録」を表示 */
      <div className="mt-6">
        <Link
          href="/books/new"
          className="inline-flex items-center px-4 py-2 border border-transparent
            shadow-sm text-sm font-medium rounded-md text-white bg-blue-600 hover:bg-blue-700"
        >
          最初の書籍を登録する
        </Link>
      </div>
    )}
  </div>
) : (
  <div className="bg-white shadow overflow-hidden sm:rounded-lg">
    <ul className="divide-y divide-gray-200">
      {books.map((book) => ( /* 配列を map で展開してリスト要素を生成 */
        <li key={book.id}> /* key は React がリスト要素を識別するために必要 */
          <Link
            href={`/${books}/${book.id}`} // クリックで詳細ページへ
            className="block hover:bg-gray-50 transition-colors duration-150"
          >
            <div className="px-4 py-4 sm:px-6">
              <div className="flex items-center justify-between">
                <div className="flex-1 min-w-0">
                  <p className="text-sm font-medium text-blue-600 truncate">
                    {book.title}
                  </p>
                  <p className="mt-1 text-sm text-gray-500">
                    {book.author || "著者不明"}
                    {book.publisher && ` / ${book.publisher}`} /* 出版社があれば
併記 */
                  </p>
                </div>
                <div className="ml-4 flex flex-col items-end space-y-1">
                  <span
                    className={`inline-flex items-center px-2.5 py-0.5 rounded-
full text-xs font-medium ${getStatusColor(book.status)} `}
                  >
                    {getStatusLabel(book.status)}
                  </span>
                  {book.rating && (
                    <span className="text-yellow-500 text-xs">
                      {renderStars(book.rating)}
                    </span>
                  )}
                </div>
              </div>
            </div>
          </li>
        )}
      </ul>
    </div>
  )
}

```

```

        </div>
      </Link>
    </li>
  )}
</ul>
</div>
)}
</div>
);
}

```

検索・フィルタ機能のポイント:

- Supabase の `ilike` メソッドは、PostgreSQL の `ILIKE` 演算子に対応しており、大文字小文字を区別しない部分一致検索を行います。`%` はワイルドカードで、`%React%` は「React」を含むすべての文字列にマッチします。
- `.or()` メソッドを使うことで、「タイトルに含まれる または 著者名に含まれる」という OR 条件を設定できます。
- URL の検索パラメータ (`searchParams`) を使うことで、検索状態が URL に反映されます。ブラウザの戻る/進むボタンで検索状態を行き来したり、検索結果の URL を他の人と共有したりできます。

5. ソート機能

書籍一覧をさまざまな基準で並び替えられるソートコンポーネントを作成します。

ファイル: `components/SortSelect.tsx`

▼ このコードがやること（先に日本語で）：一覧の並び順を切り替えるセレクトボックスを作ります。選択肢の値は `"title-asc"` のように「カラム名-昇順/降順」をハイフンでつないだ形式にしておき、選ばれたら2つに分解して URL の `?sort=...&order=...` に書き込みます。検索バーと同じく、並び順も URL に持たせることで共有や戻る操作に対応できます。初心者は「選択 → 値を分解 → URL を更新 → サーバーがその順番で取り直す」という流れを押さえてください。詳細はコード内のコメントにあります。

```

"use client"; // セレクト変更時に router.push を呼ぶのでクライアントコンポーネント

import { useRouter, useSearchParams, usePathname } from "next/navigation"; // ルーティン
          グ関連フック

// -----
// ソート選択コンポーネント
// -----
// 書籍一覧の並び順を変更するためのセレクトボックスです。
// 以下のソート基準に対応しています:
// - 登録日（新しい順/古い順）
// - タイトル（昇順/降順）
// - 評価（高い順/低い順）
// -----
export default function SortSelect() {
  const router = useRouter();
  const searchParams = useSearchParams();
  const pathname = usePathname();

  // -----
  // 現在のソート状態をURLから取得
  // -----
  // URL に sort/order が無ければデフォルト値を使う。
  // -----
  const currentSort = searchParams.get("sort") || "created_at"; // デフォルトは登録日
  const currentOrder = searchParams.get("order") || "desc"; // デフォルトは降順
  const currentValue = `${currentSort}-${currentOrder}`; // select の value 形
  式に変換

  // -----
  // ソート変更ハンドラ
  // -----
  // セレクトボックスの値が変更されたときに呼ばれます。
  // 値は "カラム名-昇順/降順" の形式（例: "title-asc"）
  // になっており、これを分解して URL パラメータに
  // 設定します。
  // -----
  const handleSortChange = (e: React.ChangeEvent<HTMLSelectElement>) => {
    const value = e.target.value; // 例: "rating-desc"
    const [sort, order] = value.split("-"); // ["rating", "desc"] に分解

    const params = new URLSearchParams(searchParams.toString()); // 現在のクエリをコピー
    params.set("sort", sort); // sort を上書き
    params.set("order", order); // order を上書き

    router.push(`${pathname}?${params.toString()}`); // 新しい URL に遷移
  };

  // -----

```

```
// レンダリング
// -----
return (
  <div className="flex items-center justify-end mb-4">
    <label
      htmlFor="sort"
      className="text-sm text-gray-500 mr-2"
    >
      並び順:
    </label>
    <select
      id="sort"
      value={currentValue} // URL の値を反映
      onChange={handleSortChange} // 変更時に URL を更新
      className="block w-48 py-1.5 px-3 border border-gray-300 bg-white rounded-md
shadow-sm focus:outline-none focus:ring-blue-500 focus:border-blue-500 text-sm"
    >
      <option value="created_at-desc">登録日（新しい順） </option>
      <option value="created_at-asc">登録日（古い順） </option>
      <option value="title-asc">タイトル（A→Z） </option>
      <option value="title-desc">タイトル（Z→A） </option>
      <option value="rating-desc">評価（高い順） </option>
      <option value="rating-asc">評価（低い順） </option>
    </select>
  </div>
);
}
```

ソート機能のポイント:

- Supabase の `.order()` メソッドは、PostgreSQL の `ORDER BY` 句に対応しています。第1引数にカラム名、第2引数のオブジェクトで `ascending: true` を指定すると昇順、`ascending: false`（または省略）で降順になります。
- ソート条件もURLの検索パラメータに反映されるため、検索・フィルタと同様にブラウザの履歴管理やURL共有が可能です。
- 検索・フィルタとソートは独立して動作します。例えば「未読」フィルタをかけた状態で「評価（高い順）」にソートすると、「未読かつ評価の高い順」で表示されます。

6. 全機能の動作確認

ここまでで実装した CRUD 全操作と検索・フィルタ・ソート機能の動作を確認します。以下の手順に沿って、各機能を順番にテストしてください。

6-1. 新規登録のテスト

1. ブラウザで `http://localhost:3000/books` にアクセスします
2. 右上の「新規登録」ボタンをクリックします
3. `http://localhost:3000/books/new` に遷移したことを確認します
4. 以下の情報を入力します: - タイトル: `React実践ガイド` - 著者: `山田太郎` - 出版社: `技術評論社` - 出版日: `2024-01-15` - 評価: `★★★★☆ (4)` - ステータス: `読書中` - メモ: `コンポーネント設計の章が特に参考になった`
5. 「登録する」ボタンをクリックします
6. 期待される結果: 一覧ページにリダイレクトされ、登録した書籍が一覧に表示されていること
7. 同様に、もう2冊登録します: - タイトル: `TypeScript入門`, 著者: `鈴木花子`, ステータス: `未読`, 評価: `3` - タイトル: `Next.js実践`, 著者: `田中一郎`, ステータス: `読了`, 評価: `5`

6-2. 詳細表示のテスト

1. 一覧ページで「React実践ガイド」をクリックします
2. 詳細ページに遷移したことを確認します
3. 期待される結果: - URL が `/books/{書籍ID}` の形式であること - タイトル「React実践ガイド」がヘッダーに表示されていること - 著者「山田太郎」がヘッダーの下に表示されていること - ステータス「読書中」のバッジが青色で表示されていること - 出版社「技術評論社」が表示されていること - 出版日「2024年1月15日」が日本語形式で表示されていること - 評価「★★★★☆」が黄色い星で表示されていること - メモの内容が灰色の背景内に表示されていること - 「編集する」ボタン（青色）と「削除する」ボタン（赤色）が表示されていること
4. 「一覧に戻る」リンクをクリックして、一覧ページに戻れることを確認します

6-3. 編集のテスト

1. 詳細ページに戻り、「編集する」ボタンをクリックします
2. 編集ページに遷移したことを確認します
3. 期待される結果: - URL が `/books/{書籍ID}/edit` の形式であること - フォームの各フィールドに既存のデータが入った状態で表示されていること
 - タイトル欄に「React実践ガイド」
 - 著者欄に「山田太郎」
 - 出版社欄に「技術評論社」
 - 出版日が「2024-01-15」
 - 評価が「★★★★☆ (4)」
 - ステータスが「読書中」
 - メモ欄に「コンポーネント設計の章が特に参考になった」
 - ボタンのテキストが「更新する」であること

4. 以下の変更を行います: - 評価を「★★★★★ (5)」に変更 - ステータスを「読了」に変更 - メモに「最後まで読了。全体的に素晴らしい内容だった」を追加
5. 「更新する」ボタンをクリックします
6. **期待される結果:** 詳細ページにリダイレクトされ、更新された情報が反映されていること - 評価が「★★★★★」に変わっていること - ステータスが「読了」（緑色のバッジ）に変わっていること - メモが更新されていること

6-4. 検索・フィルタのテスト

1. 一覧ページに戻ります
2. 検索バーに「React」と入力します
3. **期待される結果:** 「React実践ガイド」のみが表示されること - URL に `?q=React` が含まれていること - 「1件の書籍が見つかりました（検索:「React」）」と表示されること
4. 検索バーをクリアし、ステータスフィルタで「未読」を選択します
5. **期待される結果:** - ステータスが「未読」の書籍のみが表示されること - URL に `?status=unread` が含まれていること
6. 「クリア」ボタンをクリックして検索条件をリセットします
7. **期待される結果:** 全書籍が表示されること

6-5. ソートのテスト

1. ソートのセレクトボックスで「評価（高い順）」を選択します
2. **期待される結果:** 評価の高い書籍から順に表示されること
3. 「タイトル（A→Z）」を選択します
4. **期待される結果:** タイトルのアルファベット/五十音順に表示されること

6-6. 削除のテスト

1. 一覧ページから「TypeScript入門」をクリックして詳細ページに移動します
2. 「削除する」ボタンをクリックします
3. **期待される結果:** 確認ダイアログが表示され、「本当に「TypeScript入門」を削除しますか？この操作は取り消せません。」というメッセージが表示されること
4. まず「キャンセル」をクリックします
5. **期待される結果:** 何も変化せず、詳細ページが表示されたままであること
6. 再度「削除する」ボタンをクリックし、今度は「OK」をクリックします
7. **期待される結果:** - 一覧ページにリダイレクトされること - 「TypeScript入門」が一覧から消えていること - 残りの書籍が正しく表示されていること

7. トラブルシューティング

開発中によく遭遇するエラーとその対処法をまとめます。

7-1. params の取得でエラーが発生する

エラーメッセージ:

```
Error: Cannot read properties of undefined (reading 'id')
```

または

```
TypeError: params.then is not a function
```

原因と対処法:

Next.js のバージョンによって `params` の扱いが異なります。

- Next.js 15 以降: `params` は `Promise` として渡されるため、`await params` で解決する必要があります。
- Next.js 14 以前: `params` は通常のオブジェクトとして渡されるため、直接 `params.id` でアクセスします。

```
// Next.js 15 以降
type Props = {
  params: Promise<{ id: string }>; // Promise でラップされている
};

export default async function Page({ params }: Props) {
  const { id } = await params; // await が必要
  // ...
}

// Next.js 14 以前
type Props = {
  params: { id: string }; // ただのオブジェクト
};

export default async function Page({ params }: Props) {
  const { id } = params; // await 不要
  // ...
}
```

自分のプロジェクトの Next.js バージョンは `package.json` で確認できます。

```
cat package.json | grep next    # package.json から "next" を含む行を抜き出して表示
```

7-2. UPDATE / DELETE が反映されない

症状: UPDATE や DELETE を実行してもエラーは出ないが、データが変更されていない。

原因1: RLS (Row Level Security) ポリシーが設定されていない、または不適切

Supabase では RLS がデフォルトで有効になっています。テーブルに適切なポリシーが設定されていない場合、操作が静かに無視されることがあります。

対処法: Supabase のダッシュボードで RLS ポリシーを確認します。

```
-- Supabase SQL Editor で実行
-- 現在のポリシーを確認
SELECT * FROM pg_policies WHERE tablename = 'books'; -- システムビューから books テーブル
用のポリシーを一覧表示
```

開発中は、以下のような全許可ポリシーを設定できます（本番環境では適切な認証ベースのポリシーに変更してください）。

▼ このコードがやること（先に日本語で）: Supabase の SQL Editor で実行する、開発用の「全部許可」ルール（ポリシー）を作る SQL です。Supabase はセキュリティ機能（RLS）により、ルールが無いと読み書きが静かに無視されてしまうため、学習中だけ「誰でも全操作OK」のルールを置いて動くようにします。USING (true) と WITH CHECK (true) の true が「常に許可」を意味します。初心者は「これは開発専用で、本番では必ず認証ベースのルールに置き換える」点だけ覚えておいてください。

```
-- すべての操作を許可するポリシー（開発用）
CREATE POLICY "Allow all operations" ON books -- ポリシー名を "Allow all operations" と
して books テーブルに作成
  FOR ALL -- SELECT/INSERT/UPDATE/DELETE すべて
に適用
  USING (true) -- 既存行へのアクセス条件: 常に true (全許
可)
  WITH CHECK (true); -- 新規追加/更新時のチェック条件: 常に
true (全許可)
```

原因2: .eq() の条件が合っていない

UPDATE や DELETE で .eq("id", bookId) の bookId が正しくない可能性があります。

対処法: コンソールログで値を確認します。


```

console.log("Updating book with ID:", bookId); // 渡ってきた bookId を確認
const { data, error } = await supabase
  .from("books")
  .update({ title: "..." })
  .eq("id", bookId)
  .select(); // .select() を付けて更新後のデータを返してもらう
console.log("Update result:", { data, error }); // data が [] なら該当 ID 無し、null なら RLS で弾かれている可能性

```

原因3: `router.refresh()` を呼んでいない

Next.js のサーバーコンポーネントはデータをキャッシュするため、データベースを更新しただけでは画面に反映されません。

対処法: `router.push()` の後に `router.refresh()` を呼びます。

```

router.push("/books");
router.refresh(); // サーバーコンポーネントのキャッシュを更新（最新データで再取得）

```

Server Action を使っている場合は `revalidatePath` を使う: `revalidatePath("/books")` を呼ぶと、指定した URL のキャッシュが破棄され、次のアクセスで最新データが取れます。Server Action 内では `router.refresh` が呼べないので、こちらを使います。

7-3. リダイレクトが動作しない

症状: `router.push()` を呼んでもページが遷移しない。

原因1: サーバーコンポーネント内で `useRouter` を使っている

`useRouter` はクライアントコンポーネント専用のフックです。サーバーコンポーネント内では使用できません。

対処法: サーバーコンポーネントでリダイレクトしたい場合は `redirect()` 関数を使います。

```

// サーバーコンポーネント内でのリダイレクト
import { redirect } from "next/navigation"; // サーバー側遷移用の関数

export default async function Page() {
  // 何らかの条件でリダイレクト
  redirect("/books"); // この行で関数全体が中断され、ブラウザは /books に飛ぶ
}

```

原因2: `"use client"` ディレクティブの付け忘れ

`useRouter` や `useState` などの React フックを使用するコンポーネントには `"use client"` ディレクティブが必要です。

```
"use client"; // ファイルの最初に追加（最上行・1 行目に書く）

import { useRouter } from "next/navigation";
// ...
```

原因3: try-catch の catch ブロック内で処理が止まっている

エラーが発生して catch ブロックに入ると、リダイレクト処理が実行されません。コンソールでエラーを確認してください。

```
try {
  // ... Supabase 操作
  router.push("/books"); // エラーが発生するとここまで到達しない
} catch (err) {
  console.error("Error:", err); // コンソールでエラーを確認
}
```

7-4. RLS 関連のエラー

エラーメッセージ:

```
{
  code: "42501",
  message: "new row violates row-level security policy for table \"books\""
}
```

原因: RLS ポリシーが書き込み操作を許可していません。

対処法:

- Supabase ダッシュボードの「Authentication」>「Policies」で、`books` テーブルのポリシーを確認します。
- 開発段階では、以下の SQL で全操作を許可できます。

▼このコードがやること（先に日本語で）：先ほどの「全部許可」ポリシーを作り直す SQL です。同じ名前のポリシーが既にあったら `CREATE` でエラーになるため、先に `DROP POLICY IF EXISTS` で「あれば消す」を実行してから作り直しています（`IF EXISTS` のおかげで、無くてエラーになりません）。初心者は「ポリシーを入れ替えたいときは、いったん消してから作り直す」という2段階のやり方として覚えておけば大丈夫です。

```

-- 既存のポリシーを削除（必要に応じて）
DROP POLICY IF EXISTS "Allow all operations" ON books;    -- 同名ポリシーがあれば削除（IF
EXISTS でエラー回避）

-- 全操作許可ポリシーを作成
CREATE POLICY "Allow all operations" ON books              -- ポリシーを再作成
  FOR ALL                                                  -- すべての操作対象
  USING (true)                                             -- 全行を許可
  WITH CHECK (true);                                     -- 全書き込みを許可

```

3. 本番環境では、認証されたユーザーのみが操作できるようにポリシーを設定します。

▼ このコードがやること（先に日本語で）：本番向けの、安全なルール（ポリシー）の例です。ログイン済みのユーザー（`authenticated`）だけを対象にし、さらに `auth.uid() = user_id` で「自分が登録したデータだけ操作できる」ように制限します（`auth.uid()` は今ログインしている人のIDです）。初心者は「本番では『誰でもOK』ではなく『ログイン済みかつ自分のデータだけ』に絞る」という考え方を押さえてください。なお、この例は `books` テーブルに `user_id` 列がある前提で、本章の現段階では未実装なので、今は開発用ポリシーを使います。

```

-- 認証済みユーザーのみ操作を許可
CREATE POLICY "Authenticated users can manage books" ON books    -- ポリシー名は説明的に
  FOR ALL                                                         -- すべての操作
  TO authenticated                                                -- 認証済みユーザー（ロ
-- ル authenticated）のみ
  USING (auth.uid() = user_id)                                    -- 既存行の所有者チェッ
ク
  WITH CHECK (auth.uid() = user_id);                             -- 書き込み時の所有者チ
ェック

```

注意: 上記の本番用ポリシーは、`books` テーブルに `user_id` カラムがあり、ユーザーごとにデータを分離する場合の例です。このチュートリアル of 現段階では `user_id` カラムは未実装なので、開発用の全許可ポリシーを使用してください。

7-5. フォームの初期値が反映されない

症状: 編集ページでフォームを開いても、フィールドが空になっている。

原因: `useState` の初期値が正しく設定されていない可能性があります。

対処法:

1. 編集ページで `initialData` が正しく渡されているか確認します。

```
// app/books/[id]/edit/page.tsx
console.log("Book data from DB:", book); // DB から取れた生データ
console.log("Initial data for form:", initialData); // フォームに渡す整形後データ
```

2. `BookForm` コンポーネントで `initialData` を受け取れているか確認します。

```
// components/BookForm.tsx
console.log("Received initialData:", initialData); // 受け取った props
console.log("Form state:", formData); // 内部状態
```

3. `null` 値の扱いに注意します。データベースから取得した値が `null` の場合、フォームの入力フィールドが uncontrolled になる可能性があります。

```
// null を空文字に変換
const initialData = {
  title: book.title ?? "", // null の場合は空文字 (?? は null/undefined のときだけ右の値)
  author: book.author ?? "", // null の場合は空文字
  // ...
};
```

8. 全体のページ遷移図

最後に、書籍管理アプリ全体のページ遷移を確認します。以下の図は、ユーザーがアプリ内でどのようにページ間を移動するかを示しています。

書籍管理アプリ - ページ遷移図



主要な遷移パス

- 一覧 → 新規登録 → 一覧 (登録完了/キャンセル)
- 一覧 → 詳細 → 編集 → 詳細 (更新完了/キャンセル)
- 詳細 → 一覧 (「一覧に戻る」/ 削除完了)

この遷移図から、アプリの主要な導線を確認できます。

- 一覧 → 新規登録 → 一覧:** 新しい書籍を登録する流れ。登録完了後は一覧ページに戻り、登録した書籍がリストに追加されていることを確認できます。
- 一覧 → 詳細 → 編集 → 詳細:** 書籍の情報を編集する流れ。一覧から詳細を確認し、「編集する」ボタンで編集ページへ。編集完了後は詳細ページに戻り、更新内容を確認できます。
- 一覧 → 詳細 → 削除 → 一覧:** 書籍を削除する流れ。詳細ページで「削除する」ボタンを押すと確認ダイアログが表示され、確認後に削除が実行されて一覧ページに戻ります。

まとめ

この章では、以下の機能を実装しました。

機能	ファイル	説明
詳細表示	<code>app/books/[id]/page.tsx</code>	書籍の全情報を表示するページ
編集	<code>app/books/[id]/edit/page.tsx</code>	既存データをフォームに表示して更新
削除	<code>components/DeleteButton.tsx</code>	確認ダイアログ付きの削除ボタン
フォーム（共通）	<code>components/BookForm.tsx</code>	新規登録と編集の両方に対応
検索・フィルタ	<code>components/SearchBar.tsx</code>	テキスト検索とステータスフィルタ
ソート	<code>components/SortSelect.tsx</code>	複数のソート基準に対応

これで、書籍管理アプリの CRUD 機能がすべて揃いました。ユーザーは書籍の登録、表示、編集、削除を一通り行えるようになり、検索やフィルタ、ソートで効率的にデータを管理できます。

次の章では、認証機能を追加してユーザーごとにデータを分離する方法を学びます。